

Nmap - Network Security Scanner

Dominik Jurkas

Faculty of Informatics and Information Technologies (FIIT)

Slovak University of Technology in Bratislava (STU)

Bratislava, Slovakia

xjurkas@stuba.sk

Abstract—Nmap (Network Mapper) is powerful and popular open-source security tool widely used for network discovery and security auditing. System and network administrators can use it to determine which hosts are active, which ports are open, which services are running, and which operating systems are installed on a particular network. Nmap offers full understanding into network configurations and potential vulnerabilities by utilizing a range of scanning techniques and the adaptable Nmap Scripting Engine (NSE). This project examines the fundamental features of Nmap, how to install it on various major operating systems, and real-world examples of common use cases like host discovery, port scanning, and OS and service detection. The assessment points out Nmap's usefulness, adaptability, and function as an essential tool for network security administration. The experiments are supported by thorough documentation of commands, outputs, and configurations to help in replication and comprehension for users in the future.

I. INTRODUCTION

Network Mapper (Nmap) is one of those tools that every security professional knows and trusts. Imagine being able to scan an entire network to find out which devices are active, what services they are running, which ports are open, and even what operating systems are installed—all from a single tool. That's exactly what Nmap does, and it does it incredibly well. Lets go in more detail, Nmap uses a clever engine that sends specially crafted packets over the network and analyzes the responses to build a detailed and accurate view of the target systems. What makes it even more powerful is the Nmap Scripting Engine (NSE), which lets you automate and customize scans so its like from simple network discovery to complex vulnerability detection. One of the reasons Nmap is so popular is that it runs smoothly on all the big operating systems (Windows, Linux, and macOS) making it accessible to almost anyone interested in network security. Whether you're a beginner or an expert, knowing how to use Nmap is essential if you want to understand secure networks effectively.

A. Motivation

This project was undertaken as part of the semestral assignment for the course Principles of Information Technologies Security. I chose Nmap because I found it fascinating after watching several online videos showcasing the surprising things that can be done with what seems like a straightforward tool. Those demonstrations sparked my curiosity and motivation to explore Nmap in depth and understand its real-world applications.

II. SETUP

In this section, we will walk through the installation process of Nmap on the three major operating systems: Linux, Windows, and macOS. Step-by-step instructions will be provided to ensure that setting up Nmap is straightforward regardless of your platform. By the end of this section, you will be able to install and verify Nmap successfully to begin your network scanning and security auditing tasks.

A. Windows

Installing Nmap on Windows is straightforward thanks to an official installer provided by the Nmap project. Follow these steps:

- 1) Visit the official Nmap download page website: <https://nmap.org/download.html> and scroll down until you see Microsoft Windows binaries or just click windows on the main page.



Fig. 1. Nmap download page

- 2) Download the latest stable Windows installer

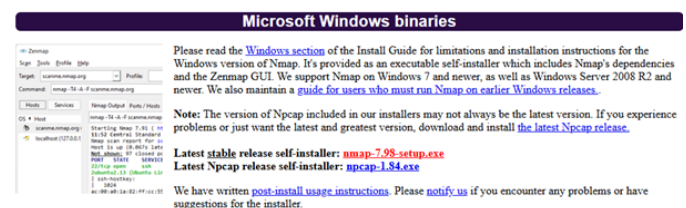


Fig. 2. Choosing Windows installer

- 3) Run the installer and follow the on-screen instructions. During installation, make sure to select the option

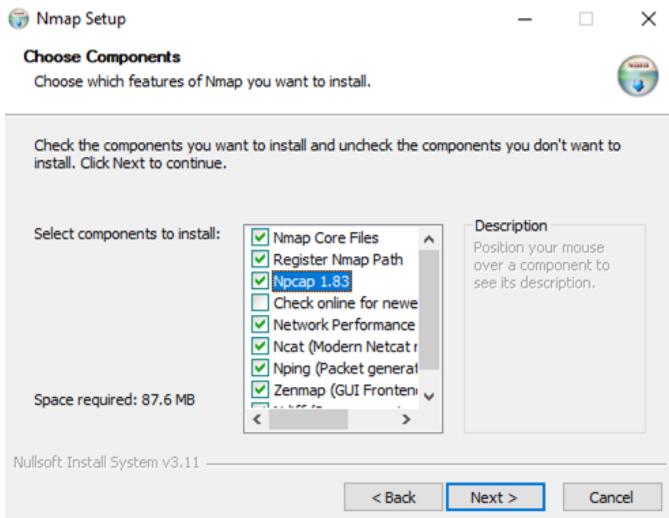


Fig. 3. Installation of Nmap and Npcap

to install Npcap, a required packet capture driver for Windows.

- 4) Once installation completes, open the Command Prompt (cmd) and type `nmap --version` to verify that Nmap is properly installed and accessible.



Fig. 4. Verify if properly installed

- 5) You can now use Nmap via command line in Windows to perform network scanning and auditing.

Npcap is essential as it allows Nmap to interact with network packets on Windows, enabling advanced scanning features. The installer usually handles this automatically.

B. Linux

Installing Nmap on Linux is simple since it is available in the default repositories of most major distributions. Follow these steps:

- 1) Update your system's package database. Open a terminal and type:

```
sudo apt update
```

- 2) Install Nmap using your package manager:
For Debian/Ubuntu:

```
sudo apt install nmap
```

For Fedora:

```
sudo dnf install nmap -y
```

For CentOS/RHEL:

```
sudo yum install nmap -y
```

For Arch/Manjaro:

```
sudo pacman -S nmap --noconfirm
```

- 3) Verify the installation by running:

```
nmap --version
```

This should display the installed version and confirm that Nmap is ready to use.



Fig. 5. Verify if Nmap successfully installed

Nmap is included in the default repositories of most Linux distributions, so installation is quick and easy. Unlike Windows, no extra drivers (such as Npcap) are required.

C. macOS

Installing Nmap on macOS can be done using Homebrew or the official installer. Both methods are straightforward and require no additional drivers.

- 1) Using Homebrew (recommended method):

If you already have Homebrew installed, open a terminal and run:

```
brew install nmap
```

After installation, verify the installation with:

```
nmap --version
```

- 2) Using the official installer:

Download the latest Nmap macOS installer from the official Nmap website: <https://nmap.org/download.html>. Open the downloaded .dmg file and follow the on-screen instructions to install Nmap.

The Homebrew method is widely used by developers and makes keeping Nmap up to date very easy. Nmap works out of the box on macOS without requiring any extra drivers.

III. GRAPHICAL USER INTERFACE: ZENMAP

Although Nmap is primarily used through the command line, it also provides an official graphical user interface (GUI) called **Zenmap**. Zenmap makes basic scans easier for beginners and offers visual tools such as topology maps and host lists. In this project, Zenmap is presented only as an additional tool. All following technical explanations (Section IV) and experiments (Section V) are performed strictly in the **command-line interface (CLI)** to ensure full control over Nmap's capabilities.

A. Zenmap Overview

Zenmap acts as a graphical front-end to Nmap. Each scan performed in Zenmap still uses the Nmap engine, and the exact command used is always visible in the “Command” bar. This helps new users understand how GUI actions translate into real Nmap commands.

Upon launching Zenmap, the main interface appears as shown in Figure 6. It includes fields for target selection, scan profiles, and multiple result tabs.

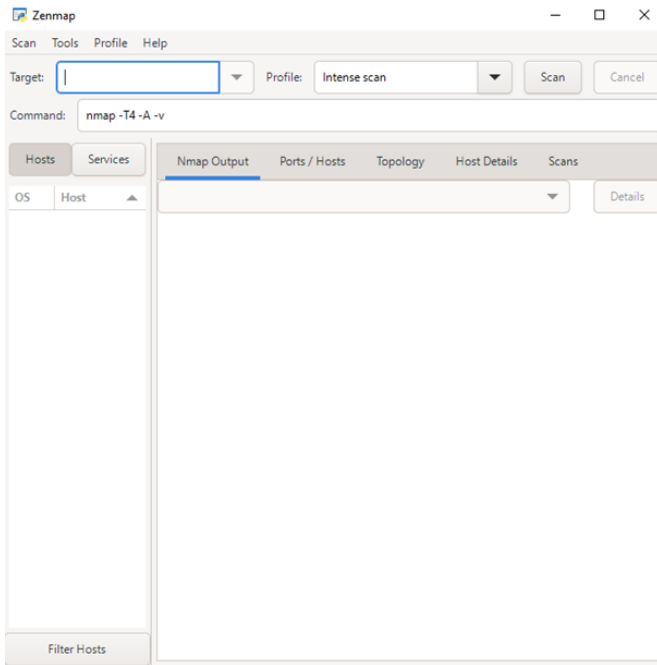


Fig. 6. Zenmap main interface after launch.

B. Performing a Scan in Zenmap

The following steps demonstrate how a basic scan can be performed using Zenmap. These steps correspond to the five included screenshots.

1) *Step 1: Entering the Target and Selecting a Profile:* In the **Target** field, the desired IP address or network is entered. A scan type is selected from the **Profile** dropdown. Zenmap automatically updates the Nmap command (Figure 7).

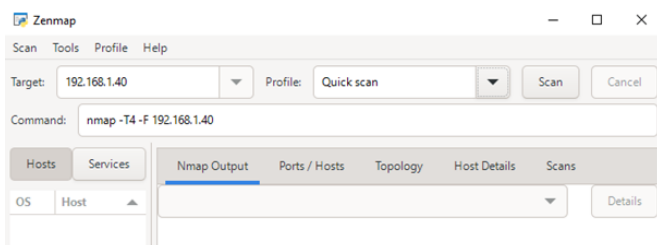


Fig. 7. Target and scan profile configured before launching the scan.

2) *Step 2: Scan Execution and Real-Time Output:* After clicking **Scan**, Zenmap displays the output exactly as Nmap would print it in a terminal. This includes host discovery, open ports, and service detection (Figure 8).

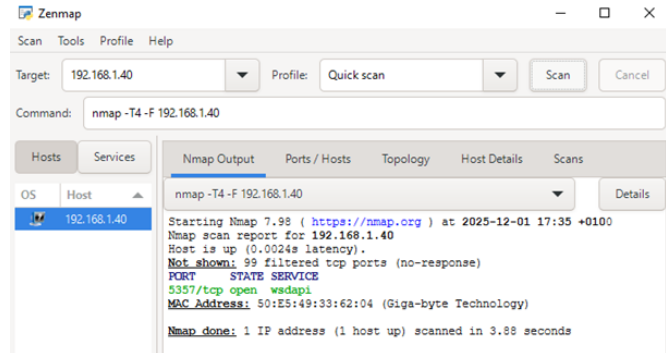


Fig. 8. Nmap Output tab showing real-time scan results.

3) *Step 3: Viewing Discovered Ports and Services:* Once the scan is completed, the **Ports / Hosts** tab provides a clear overview of all detected open ports, associated services, and versions. This view is shown in Figure 9.

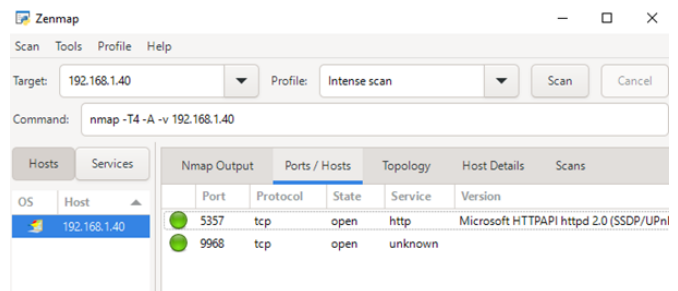


Fig. 9. Ports / Hosts tab displaying open ports and services.

4) *Step 4: Visualizing Network Topology:* Zenmap also includes a graph-based **Topology** view, illustrating the structure of the scanned network and relationships between hosts. This is a useful visualization feature not available directly in the CLI (Figure 10).

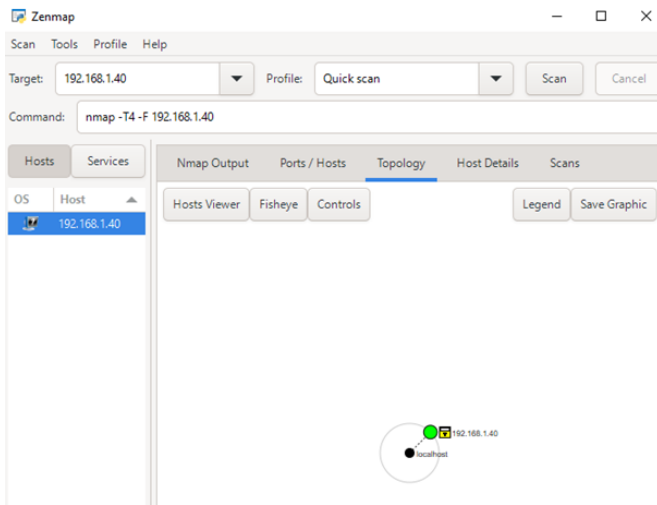


Fig. 10. Topology view showing graphical representation of the network.

C. Conclusion

Zenmap simplifies launching common Nmap scans and provides helpful visual representations for beginners. However, it does not expose the full depth of Nmap's capabilities. For this reason, all advanced explanations and experiments in this project are conducted using the command-line version of Nmap, which remains the most powerful and precise way to perform security assessments.

IV. EXPLANATION OF FUNCTIONS

Nmap's core functions are designed for flexible and comprehensive network scanning and security assessment. Each function corresponds to specific command-line options, letting users combine tasks as needed.

A. Host Discovery

Host discovery ("ping scanning") identifies which targets are alive before Nmap proceeds to port or service scanning. This avoids wasting time on offline IPs, especially in large networks. Nmap sends lightweight probes (ICMP, TCP, UDP, ARP) and considers a host up if *any* response is received. The default technique combines ICMP pings with TCP SYN/ACK probes to ports 80/443 and ARP on local networks.

1. Basic Ping Sweep: **-sn**

Performs discovery without port scanning. Useful for quick mapping, inventory, or reducing unnecessary scans.

Nmap sends ICMP echo/timestamp requests, TCP SYN to 443, TCP ACK to 80, and ARP on local Ethernet. Any reply confirms host is up.

Example:

```
nmap -sn 192.168.1.0/24

Nmap scan report for
192.168.1.10
Host is up (0.45ms latency).
MAC Address: 11:22:33:44:55:66
```

IPv6 uses ICMPv6 echo and Neighbor Discovery.

2. Skip Discovery: **-Pn**

Treats all hosts as alive and immediately performs port scanning. Necessary when firewalls block ping probes but increases scan time.

Example:

```
nmap -Pn 192.168.1.0/24
```

Use `--host-timeout` or `--max-hostgroup` to improve performance on large ranges.

3. List Scan: **-sL**

No probes sent—Nmap only lists target IPs and performs reverse DNS. Completely passive.

```
nmap -sL 192.168.1.0/24
```

Useful for verifying target ranges or collecting hostnames before active scanning.

4. TCP SYN Ping: **-PS**

Sends SYN packets; either SYN-ACK (open) or RST (closed) indicates the host is up. Common when ICMP is filtered.

```
nmap -PS22,80,443 -sn
203.0.113.0/24
```

5. TCP ACK Ping: **-PA**

Sends ACK packets; a returned RST confirms the host is alive.

```
nmap -PA80,443 -sn
203.0.113.10
```

Works well on firewalls that block SYN but allow ACK traffic.

6. UDP Ping: **-PU**

Sends empty UDP packets; ICMP "Port Unreachable" indicates host is up.

```
nmap -PU53,161 -sn
203.0.113.0/24
```

Useful where UDP is allowed but ICMP/TCP are filtered.

7. ICMP Variants: **-PE, -PP, -PM**

Alternative ICMP types that may bypass strict echo filtering.

```
nmap -PE -sn
192.168.1.0/24
nmap -PP -PM -sn
192.168.1.0/24
```

8. IP Protocol Ping: **-PO**

Sends packets with specific IP protocol numbers (e.g., ICMP, IGMP). Useful on networks where port-based probes are blocked.

```
nmap -PO1,6,17 192.168.1.10
```

9. ARP Ping (Automatic on Local Nets)

Nmap prefers ARP on local IPv4 networks since it is the most reliable discovery method. ARP replies directly confirm host presence.

```
nmap -sn 192.168.1.0/24
```

Disable with `--disable-arp-ping`.

10. Ignore Spoofed RSTs: `--discovery-ignore-rst`

Avoids false “host up” results on firewalls that spoof RST packets.

```
nmap --discovery-ignore-rst  
-PS80 -sn 203.0.113.0/24
```

B. Port Scanning

Port scanning determines which TCP/UDP services are reachable on discovered hosts. Nmap classifies each port as **open**, **closed**, **filtered**, or **open—filtered**, depending on responses and firewall behavior. By default, Nmap scans the top 1,000 TCP ports, but users can specify ranges and combine scan types for speed or stealth.

1. TCP SYN Scan (`-sS`)

The default and most common scan when raw sockets are available. SYN scan sends a SYN and infers the port state from the reply without completing the handshake.

- SYN-ACK = **open** (Nmap replies RST)
- RST = **closed**
- No response / ICMP errors = **filtered**

Example:

```
nmap -sS 192.168.1.10  
nmap -sS -p- 192.168.1.10
```

SYN scan is fast, stealthy, and works well with evasion and timing features.

2. TCP Connect Scan (`-sT`)

Used when raw packets are unavailable. Nmap relies on the system `connect()` call.

- Successful connection = **open**
- Connection refused = **closed**
- Timeout/ICMP = **filtered**

Example:

```
nmap -sT 192.168.1.10
```

More detectable than SYN scan because it completes full handshakes.

3. UDP Scan (`-sU`)

Identifies UDP services such as DNS, SNMP, and NTP. UDP scanning is slower due to sparse replies and ICMP rate limiting.

- ICMP Port Unreachable = **closed**
- UDP response = **open**
- No response = **open—filtered**

Example:

```
nmap -sU -p 53,123,161  
192.168.1.10
```

Often combined with TCP:

```
nmap -sS -sU -p T:1-1024,  
U:1-1024 host
```

4. NULL, FIN, and Xmas Scans (`-sN`, `-sF`, `-sX`)

Send unusual TCP flag combinations to identify hosts that follow RFC 793 behavior. Useful for firewall and IDS testing.

Interpretation:

- RST = **closed**
- No response = **open—filtered**

Example:

```
nmap -sN 192.168.1.10  
nmap -sF -p 1-1024 host
```

Some OSes (e.g., older Windows) do not implement RFC 793 correctly, reducing reliability.

5. Port Selection and Combined Scans (`-p`, multiple `-s*`)

The `-p` option specifies ports using lists, ranges, or protocol prefixes (e.g., `T:22,U:53`). Multiple scan types may run simultaneously.

Examples:

```
# All TCP ports  
nmap -sS -p- host
```

```
# Mixed TCP + UDP  
nmap -sS -sU -p T:22,  
80,U:53,161 host
```

Notes: Scanning all 65,535 ports is thorough but slow; `--top-ports` focuses only on the most common ones.

C. Service and Version Detection

Service and version detection identifies the exact application and software version running on open ports by sending protocol-specific probes and matching responses against Nmap’s `nmap-service-probes` database. This converts raw port data into actionable intelligence (e.g., nginx vs. Apache, OpenSSH vs. Dropbear) and enables targeted vulnerability assessment. TCP yields the most reliable results; UDP is supported but less consistent.

1. Basic Version Detection (`-sV`)

`-sV` probes all open ports discovered during scanning, sending protocol-appropriate requests and matching banners or responses to known fingerprints.

Example:

```
nmap -sV 192.168.1.10  
nmap -sV -p 22,80,443  
192.168.1.10
```

Typical Output:

```
22/tcp open ssh
OpenSSH 8.2p1
80/tcp open http
nginx 1.18.0
```

Notes: Default intensity is 7 (of 9). Lower levels probe less aggressively; level 9 tries all probes and may affect unstable services.

2. Aggressive Version Detection (`--version-intensity`)

Controls probe aggressiveness (1–9). Higher levels increase accuracy but may trigger IDS alerts or cause service instability.

Example:

```
nmap -sV --version-intensity
9 192.168.1.10
```

Notes: `--version-light` uses a lightweight profile (intensity 2).

3. Specific Probe Selection (`--version-probes`)

Allows running only selected probe types (e.g., `http`, `ssl`, `ssh2`) for troubleshooting or identifying unusual services.

Example:

```
nmap -sV --version-probes
http,ssl -p 8080 192.168.1.10
```

4. UDP Version Detection

UDP detection is supported but less reliable due to limited responses. Nmap sends protocol-specific payloads such as DNS queries, SNMP GET requests, or TFTP reads.

Example:

```
nmap -sU -sV 192.168.1.10
```

Output:

```
53/udp open domain 9.16.1
161/udp open snmp SNMPv2c
```

5. SSL/TLS Detection and Script Integration

Nmap includes TLS-aware probes that reveal certificate and handshake details and integrate with NSE scripts for deeper analysis.

Example:

```
nmap -sV -p 443
192.168.1.10
nmap --script ssl-cert
-p 443 192.168.1.10
```

Output:

```
443/tcp https Apache/2.4.41
| ssl-cert: Subject:
CN=example.com
```

Notes: TLS 1.3 servers may expose fewer details; `ssl-enum-ciphers` provides cipher suite enumeration.

D. OS Detection

Nmap's OS detection identifies a target's operating system by analyzing subtle TCP/IP stack behaviors such as initial TTL values, window sizes, IP ID sequences, TCP options, and ICMP responses. These low-level fingerprints help determine OS family, version, and device type, guiding vulnerability analysis and exploitation strategy.

Nmap compares probe responses against its `nmap-os-db` (5,000+ fingerprints), typically achieving over 90% accuracy on responsive hosts. Probes include TCP SYN/ACK/FIN/NULL/Xmas packets and ICMP echo, timestamp, and netmask requests. Accuracy depends on firewall filtering and network interference.

1. Basic OS Detection (`-O`)

`-O` sends a full fingerprinting probe set and matches results against the OS database.

Example:

```
nmap -O 192.168.1.10
nmap -sS -O 192.168.1.0/24
```

Typical Output:

```
Running: Linux 4.X|5.X
OS details: Linux 5.4.0-42-generic
```

Notes: Requires raw sockets. `--osscan-limit` skips hosts unlikely to yield results; `--osscan-guess` enables looser matching.

2. OS and Version Detection (`-A`)

`-A` combines OS detection with version scanning, default NSE scripts, and traceroute for full host profiling.

Example:

```
nmap -A 192.168.1.10
```

Output Example:

```
OS details: Linux 4.15 - 5.8
TRACEROUTE (using port 22/tcp)
```

Notes: More noisy and slower than `-O`; may trigger alerts.

3. Refining Match Behavior (`--osscan-match`)

Advanced controls improve accuracy or speed by limiting match criteria.

Useful Options:

- `--max-os-tries`: limit retries
- `--osscan-match` any|distance|latency

Example:

```
nmap -O --max-os-tries 1
--osscan-match distance 192.168.1.0/24
```

Helpful for cloud environments with many similar VMs.

4. IPv6 OS Detection

Nmap supports IPv6 OS detection using ICMPv6 and TCPv6 probes.

Example:

```
nmap -6 -O 2001:db8::1
```

Accuracy may be reduced where ICMPv6 is heavily filtered.

5. Troubleshooting Detection Failures

Filtering or packet normalization can obscure fingerprints. Diagnostic flags include:

- `--reason`: show classification causes
- `--packet-trace`: log sent/received packets
- `--stats-every 10s`: progress updates

Example:

```
nmap -O --packet-trace  
--reason 192.168.1.10
```

Normalization of TTL or window size by firewalls often reduces accuracy.

E. Nmap Scripting Engine (NSE)

The Nmap Scripting Engine (NSE) extends Nmap by running Lua-based scripts for tasks such as service enumeration, authentication testing, vulnerability detection, and protocol analysis. NSE operates after port and version detection, using discovered services to perform targeted checks. With over 700 scripts across categories such as `vuln`, `auth`, `brute`, and `discovery`, NSE enables deep automated analysis while integrating cleanly with Nmap's output formats.

Scripts run in a sandboxed Lua environment with access to Nmap data, sockets, and libraries for HTTP, SMB, DNS, SNMP, and more. Each script specifies required ports and categories (`safe`, `intrusive`), enabling parallel execution only when applicable.

1. Default Script Scan (`-sC`)

`-sC` runs Nmap's set of *default safe* scripts, suitable for general reconnaissance and automatically chosen based on open ports.

Example:

```
nmap -sC 192.168.1.10
```

Output Example:

```
| ssh-hostkey:  
|   256 96:ce:cd:... (RSA)  
| http-title: Go ahead and ScanMe!
```

Default scripts avoid disruptive behavior. View them using `--script-help default`.

2. Category-Based Script Selection

Scripts are grouped by function (e.g., `vuln`, `auth`, `brute`, `safe`, `discovery`). Running a category executes all applicable scripts.

Example:

```
nmap --script vuln 192.168.1.10  
nmap --script "discovery,safe"
```

192.168.1.0/24

Output Example:

```
| ssh-brute:  
|_ Found valid credentials
```

Intrusive categories may cause service disruption.

3. Specific Scripts and Arguments

Individual scripts or patterns (e.g., `http-*`) can be selected. `--script-args` customizes behavior (credentials, paths, user agents, etc.).

Example:

```
nmap --script http-title --script-args  
http.useragent=MyUA 192.168.1.10  
nmap --script ssh-brute --script-args  
userdb=users.txt,passdb=pw.txt -p22
```

Arguments may also be loaded via `--script-args-file`.

4. Vulnerability Detection Scripts (`vuln`)

The `vuln` category includes CVE checks for services such as SMB, HTTP, TLS, DNS, and databases.

Example:

```
nmap --script vuln -p80,443,445  
192.168.1.10
```

Typical Output:

```
| smb-vuln-ms17-010:  
|   VULNERABLE: CVE-2017-0144 (ms17-010)
```

False positives are possible; results should be verified manually.

5. Custom Script Development

Users can build custom Lua scripts using NSE libraries (e.g., `http`, `smb`, `snmp`, `shortport`). Scripts define metadata and an action function.

Example:

```
nmap --script /path/to/myscript.nse  
192.168.1.10
```

Custom scripts placed in `~/.nmap/scripts/` load automatically. Update the script index using `--script-updatedb`.

F. Timing and Performance

Nmap's timing and performance controls regulate scan speed, concurrency, retry behavior, and timeouts. The engine adapts dynamically to latency and packet loss to maintain efficiency while avoiding network overload. Tuning is essential for both high-speed internal scans and slow, low-noise external assessments. Two main mechanisms exist: timing templates (`-T0--T5`) and fine-grained controls (e.g., `--min-rate`, `--max-parallelism`, `--host-timeout`).

1. Timing Templates (`-T0--T5`)

Templates preset dozens of parameters for common scenarios: -T0 (paranoid), -T1 (sneaky), -T3 (default), -T4 (fast), -T5 (maximum speed with minimal safety checks).

Example:

```
nmap -T0 -sS 192.168.1.0/24
# Very slow
nmap -T4 -sS 192.168.1.0/24
# Fast internal scan
```

Additional Details: Aggressive levels (-T4/-T5) increase parallelism but may trigger IDS alerts or destabilize weak hosts.

2. Rate Control (--min-rate, --max-rate)

Rate limits define minimum and maximum packet rates.

How it Works:

- --min-rate <pps> ensures a minimum packets-per-second rate.
- --max-rate <pps> caps transmission to avoid congestion.

Example:

```
nmap --min-rate 1000
-sS 192.168.1.0/24
nmap --max-rate 50
-sS 192.168.1.0/24
```

Additional Details: High --min-rate values can cause packet loss; monitor with --stats-every 10s.

3. Parallelism Control

Controls how many probes and hosts run concurrently.

How it Works:

- --max-parallelism <n> limits probes per host.
- --min-hostgroup <n> and --max-hostgroup <n> tune host batch sizes.

Example:

```
nmap --max-parallelism
50 192.168.1.0/24
```

Additional Details: Large hostgroups accelerate internal scans but require more memory; lower values improve stability on unreliable networks.

4. Timeout and Retry Control

Timeouts and retry limits prevent scans from stalling on slow or filtered hosts.

How it Works:

- --host-timeout <time> stops scanning unresponsive hosts.
- --max-retries <n> restricts probe repetitions.
- --scan-delay <t> adds delay between probes.

Example:

```
nmap --host-timeout 2m
--max-retries 1 -sS 192.168.1.0/24
```

Additional Details: --scan-delay aids stealth or bypasses rate limits but slows scans significantly.

5. Combined Tuning Examples

Advanced scans mix timing and rate controls for specific environments.

Examples:

```
# Fast internal scanning
nmap -T4 --min-hostgroup 100
-sS 192.168.0.0/16

# Stealth external scanning
nmap -T1 --scan-delay 1s
--max-retries 2 -sS scanme.nmap.org

# Large-scale Internet scanning
nmap -T5 --min-rate 10000
-n -Pn 10.0.0.0/8
```

Additional Details: Use --stats-every for progress reporting; -n skips DNS lookups to reduce scan time.

G. Firewall/IDS Evasion and Spoofing

Nmap provides several techniques to evade firewalls and intrusion detection systems by altering packet structure, timing, or apparent source. These methods exploit weaknesses in rule processing, TCP state tracking, and IDS signature matching. While useful for authorized testing, misuse is unlawful.

Effectiveness depends on the firewall type, IDS configuration, and network path conditions. Many techniques require raw socket privileges and are often combined for increased stealth.

1. Fragmentation (-f, --mtu)

Fragmentation splits Nmap probes into small IP fragments that may slip past basic filters or be reassembled only after inspection.

- -f creates 8-byte fragments; repeating it produces 16-byte fragments.
- --mtu <size> sets a custom fragment size.
- Some filters drop fragments; others reassemble before applying rules.

Example:

```
nmap -f -sS 192.168.1.10
```

Additional Details: Modern firewalls often reassemble fragments automatically. UDP fragmentation can be more effective than TCP.

2. Decoy Scanning (-D)

Decoy scanning inserts multiple spoofed source IPs to obscure the real origin of a scan.

Nmap rotates probes among decoy addresses and the attacker's own IP (marked ME). Target logs show activity from all listed IPs.

Example:

```
nmap -D RND:5 -sS
192.168.1.10
```

Additional Details: Decoys must be routable, or replies are lost. IDS systems may still detect the scan pattern.

3. Source IP/Port Spoofing (--source-ip, --source-port)

Spoofing changes the apparent origin of probes to bypass filters that permit specific IPs or ports.

- --source-ip X.X.X.X forges the sender address.
- --source-port <port> forces use of commonly whitelisted ports (e.g., 53 for DNS).
- Replies go to the spoofed IP unless routing returns them.

Example:

```
nmap --source-port 53
-sS 192.168.1.10
```

4. Idle/Zombie Scan (-sI)

Idle scanning uses a quiet third-party host (“zombie”) to proxy probes, hiding the attacker and inferring port states from IP ID increments.

- 1) Probe zombie to learn its IP ID.
- 2) Spoof zombie’s IP and send SYN to target.
- 3) Target replies to zombie.
- 4) Recheck zombie’s IP ID to infer port openness.

Example:

```
nmap -sI zombie.host 192.168.1.10
```

Additional Details: Works best with hosts that have predictable IP ID behavior. Very stealthy but slow.

5. Timing-Based Evasion (--scan-delay, -T0/-T1)

Slowing probe rates avoids IDS rules that trigger on bursts of activity.

- --scan-delay <time> enforces a fixed pause between probes.
- -T0 (paranoid) and -T1 (sneaky) send packets very slowly.

Example:

```
nmap -T1 --scan-delay 1s
-sS 192.168.1.10
```

6. Additional Techniques

Other evasion features modify packet characteristics or exploit protocol quirks:

- --data-length <n>: adds random padding.
- --badsum: sends packets with invalid checksums.
- --discovery-ignore-rst: ignores spoofed RST interference.
- --defeat-rst-ratelimit: works around RST rate limiting.

Example:

```
nmap --data-length 25
--badsum -sS 192.168.1.10
```

Additional Details: --ip-options supports source routing; --mpls adds MPLS tags. Combining several methods increases effectiveness against hardened environments.

H. Output and Reporting

Nmap provides multiple output formats for human analysis, automation, and integration with external tools. Results include host states, port and service information, OS details, NSE results, and performance statistics. The primary formats are normal text, XML, and grepable output, with additional options for demonstrations or legacy workflows. Files may be timestamped, appended, or transformed into HTML via XSL stylesheets. XML serves as the universal data format for dashboards, SIEM pipelines, and GUI tools such as Zenmap.

1. Normal Output (-oN)

Normal output is a human-readable report mirroring terminal results and is suitable for documentation or direct review.

Nmap writes complete scan information (hosts, ports, versions, OS/NSE results) into a text file. -oA <base> produces all major formats at once, while --append-output and --timestamp extend or time-stamp files.

Example Command:

```
nmap -oN scan.txt 192.168.1.10
```

Example Output:

```
Nmap scan report for 192.168.1.10
22/tcp open  ssh
80/tcp open  http
```

Additional Details: -v increases verbosity; --stats-every prints periodic progress.

2. XML Output (-oX)

XML provides structured, machine-readable data for databases, automation scripts, and GUIs.

Scan information is stored in a hierarchical <nmaprun> document containing elements for hosts, ports, OS matches, scripts, and timing. XML supports XSL transformations for HTML/PDF generation and can be merged using --append-output.

Example Command:

```
nmap -oX scan.xml 192.168.1.10
```

Example Output:

```
<nmaprun>
  <host>
    <address addr="192.168.1.10"/>
    <ports>
      <port portid="22" protocol="tcp">
        <state state="open"/>
```

```

    </port>
  </ports>
</host>
</nmaprun>

```

Additional Details: `--stylesheet` and `--webxml` enable browser-ready HTML output.

3. Grepable Output (`-oG`)

Grepable output summarizes each host on a single line, optimized for shell scripting.

Each entry lists host metadata and port states using compact fields such as `22/open/tcp//ssh/`. This allows quick extraction with `grep`, `awk`, or similar tools.

Example Command:

```
nmap -oG scan.grep 192.168.1.0/24
```

Example Output:

```
Host: 192.168.1.10
Ports: 22/open/tcp//ssh/,80/open/tcp//http/
```

Additional Details: Useful for automation even though XML is preferred for modern parsing.

4. Advanced Output Options

Several options refine output or assist in debugging:

- `--reason`: show classification reasons
- `--packet-trace`: log all packets
- `--open`: display only open ports
- `--stats-every <time>`: periodic status
- `--resume`: continue an interrupted scan

Example Command:

```
nmap --open --reason -oN filtered.txt
192.168.1.0/24
```

5. Post-Processing and Reporting

Multiple output formats can be combined for reporting, automation, or importing results into external systems.

```
nmap -oA rawscan target.com
xsltproc nmap-bootstrap.xsl rawscan.xml
> report.html
grep "open" rawscan.gnmap > open.txt
```

Additional Details: XML integrates with Zenmap, SIEM pipelines, and security dashboards.

V. EXPERIMENTS

This section presents a series of experiments designed to demonstrate the various features and functionalities of Nmap. The goal is to showcase Nmap's capabilities for network discovery, vulnerability scanning, service detection, and stealth scanning, among others. Each experiment highlights a specific function of Nmap, explaining how it can be used in real-world network security audits and penetration testing scenarios.

Ideally, security experiments should be performed in an isolated test environment (e.g., virtual machines or a separate

lab network). In this project, first 3 experiments and scans inside them were conducted only against hosts on my own home network, for which I have administrative control, so no third-party systems were affected. The fourth and fifth experiments were conducted in purposely vulnerable machine (Metasploitable 2).

A. Experiment 1: Basic Host Discovery and Port Scanning

Objective: This experiment demonstrates Nmap's ability to discover active hosts on a network and perform a basic port scan to identify open services.

Setup: Using a local network, we will perform a host discovery scan followed by a port scan to check for open services.

Expected Outcome: The scan will display the IP addresses of active hosts on the network and the open ports with their corresponding services.

Step 1: Host Discovery with `-sn`

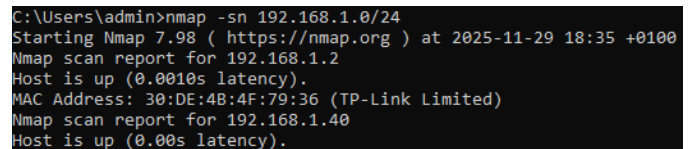
To begin the experiment, I performed a host discovery scan using the following command:

```
nmap -sn 192.168.1.0/24
```

This command sends a series of probes (ICMP Echo Request, TCP SYN to port 443, etc.) to identify which hosts are up in the `192.168.1.0/24` subnet. The result is a list of active IP addresses and their corresponding MAC addresses. This step is crucial to avoid unnecessary port scanning on inactive hosts, making the process more efficient.

Screenshot 1 and 2: Host Discovery Results

Below are screenshots of the command outputs showing the active hosts in the subnet:

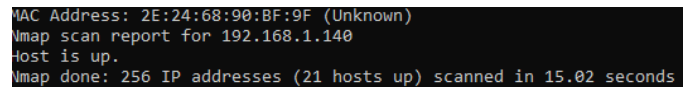


```

C:\Users\admin>nmap -sn 192.168.1.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-29 18:35 +0100
Nmap scan report for 192.168.1.2
Host is up (0.0010s latency).
MAC Address: 38:DE:4B:4F:79:36 (TP-Link Limited)
Nmap scan report for 192.168.1.40
Host is up (0.00s latency).

```

Fig. 11. Host Discovery results showing live hosts in the subnet.



```

MAC Address: 2E:24:68:90:BF:9F (Unknown)
Nmap scan report for 192.168.1.140
Host is up.
Nmap done: 256 IP addresses (21 hosts up) scanned in 15.02 seconds

```

Fig. 12. 2. Host Discovery results showing live hosts in the subnet.

The above outputs shows that hosts with IP addresses `192.168.1.2` and `192.168.1.40` are up and responding to the probes.

Step 2: Port Scanning

Once the active hosts were identified, I proceeded with a port scan to detect open ports and services on the host `192.168.1.40`. The following command was used:

```
nmap 192.168.1.40
```

Below is a screenshot of the port scanning results for the host 192.168.1.40:

```
C:\Users\admin>nmap 192.168.1.40
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-29 18:43 +0100
Nmap scan report for 192.168.1.40
Host is up (0.00013s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsdapi
9968/tcp  open  unknown
MAC Address: 50:E5:49:33:62:04 (Giga-byte Technology)

Nmap done: 1 IP address (1 host up) scanned in 17.81 seconds
```

From the output, we can see that the host has open ports 5357/tcp (wsdapi) and 9000/tcp (unknown), while 98 TCP ports are filtered, indicating that a firewall or host-based filter is blocking probes to most ports

In this experiment, we successfully used Nmap to perform a host discovery scan and a port scan. The host discovery scan revealed the active hosts in the 192.168.1.0/24 subnet, while the port scan identified open ports on one of the hosts. This demonstrates Nmap's efficiency in discovering live hosts and its ability to provide insights into the services running on those hosts.

- Port 5357/tcp: Running Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP).
- Port 9966/tcp: Identified as an unknown service.

```

C:\Users\admin\mmap -sV 192.168.1.40
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-29 18:52 +0100
Nmap scan report for 192.168.1.40
Host is up (0.00013s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
3577/tcp  open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
9686/tcp  open  unknown
           service unrecognized despite returning data. If you know the service/version, please submit a
           report at https://nmap.org/open_issue_page.html.
SF-Port9686-TCP:V=7.98;I=720-11/29;T=61=692320ZPZ-1686-pc-windows-windows
SF-Nr(GenericLines,20,"%x20;x02;x03%x20;x08;x9fH0|x08;xfbf|x08;x05;x010|x
SF-0|x0d0|xfcf|x08|x05|xf0|xb0G0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(GetReque
SF-1st,20,"%x20;x02;x03%x20;x08;x9fH0|x08;xfbf|x08;x05;x010|x0d0|x0d0|xfcf|x
SF-8|x05|x08|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(HTTPOptions,20,"%x20
SF-02;x02;x08|x08;x9fH0|x08|xfbf|x08;x05|x010|x0d0|xfcf|x08|x05|x0f0|x0
SF-dG0|x09|xec|x0eaq|xfef|fff|xfxf|fff")%nr(RTSRequest,20,"%x20;x02;x03|x02
SF-8|x05|x08|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(SSLSessionReq,20,"%
SF-0|x0d0|xfcf|x08|x05|x010|x0d0|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq
SF-ff|fff|x20|x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|xfcf|x08
SF-0|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(DNSVersionIndReqTCP,2
SF-0,"%x20;x02;x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x08|x
SF-5|xf0|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(DNSStatusRequestTCP,20,"%
SF-0|x02;x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x08|x05
SF-0|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(Help,20,"%x20;x02|x03|x02|x0
SF-8|x05|x08|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(SSLSessionReq,20,"%
SF-0|x0d0|xfcf|x08|x05|x010|x0d0|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq
SF-ff|fff|x20|x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x
SF-08|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff|x20|x02|x03|x02|x08|x9fH
SF-0|x08|xfbf|x08|x05|x010|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xf
SF-ef|fff|xfxf|fff")%nr(TerminalServerCookie,40,"%x20;x02;x03|x02|x08|x9fH0|
SF-0|x08|x05|x010|x0d0|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef|fff
SF-ff|xfxf|x20|x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x
SF-8|x05|x08|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff")%nr(TLSSessionReq,20,"%
SF-0|x02;x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x08|x05
SF-0|x0bG0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff|x20|x03|x02|x02|x08|x9fH0|x08|x
SF-fb|x08|x05|x010|x0d0|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef|fff|
SF-f|fff|x20|x02|x03|x02|x08|x9fH0|x08|xfbf|x08;x05|x010|x0d0|x0d0|xfcf|x08
SF-0|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef|fff|xfxf|fff|x20|x02|x03|x02|x08|x9fH
SF-0|x08|xfbf|x08|x05|x010|x0d0|x0d0|xfcf|x08|x05|x0f0|xb0G0|x0d9|xec|x0eaq|xfef
SF-ff|fff|xfxf|fff");
MAC Address: 50:ES:49:33:62:04 (Giga-byte Technology)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 166.73 seconds

```

This command checks for specific OS-related network signatures, such as TTL, window size, and other TCP/IP stack behaviors.

Screenshot 1: OS Detection Results

Below is the screenshot of the output from the OS detection scan:

```
C:\Users\admin>nmap -O 192.168.1.40
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-29 19:04 +0100
Nmap scan report for 192.168.1.40
Host is up (0.00023s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsdapi
9968/tcp  open  unknown
MAC Address: 50:E5:49:33:62:04 (Giga-byte Technology)
Warning: OSScan results may be unreliable because we could not find
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 10|11|2019 (92%)
OS CPE: cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_11 cpe:/o:microsoft:windows_12
Aggressive OS guesses: Microsoft Windows 10 1803 (92%), Microsoft Windows 10 1809 (87%), Microsoft Windows 10 1909 (85%), Microsoft Windows 10 2019 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 16.07 seconds
```

Fig. 15. OS Detection results for the host 192.168.1.40.

Step 2: OS Guessing with `--osscan-guess` In this step, I used the `--osscan-guess` option to allow Nmap to guess the OS if it could not make an exact match based on the detected signatures. The following command was used:

```
nmap -O --osscan-guess 192.168.1.40
```

This command enables guessing for OS when Nmap can't precisely determine the operating system.

Screenshot 2: OS Guessing Results

Here is a screenshot of the results showing Nmap's attempt to guess the OS:

```
C:\Users\admin>nmap -O --osscan-guess 192.168.1.40
Starting Nmap 7.98 ( https://nmap.org ) at 2025-11-29 19:12 +0100
Nmap scan report for 192.168.1.40
Host is up (0.00021s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
5357/tcp  open  wsdapi
9968/tcp  open  unknown
MAC Address: 50:E5:49:33:62:04 (Giga-byte Technology)
Warning: OSScan results may be unreliable because we could not find
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 10|11 (92%)
OS CPE: cpe:/o:microsoft:windows_10 cpe:/o:microsoft:windows_11 cpe:/o:microsoft:windows_12
Aggressive OS guesses: Microsoft Windows 10 1903 - 21H1 (92%), Microsoft Windows 10 1809 (87%), Microsoft Windows 10 1909 (85%), Microsoft Windows 10 2019 (85%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 11.60 seconds
```

Fig. 16. OS Guessing results for the host 192.168.1.40.

Experiment Conclusion: In this experiment, we demonstrated Nmap's ability to detect the operating system of a target host based on its network behavior. In Step 1, using the `-O` option, Nmap successfully identified the host as a Windows-based system with 92% accuracy.

In Step 2, the `--osscan-guess` option was added to allow Nmap to make a more aggressive guess when an exact match

is not available. In this case, the results remained the same (Windows, 92%), which indicates that the initial OS detection was already confident and sufficiently accurate. The guessing mode therefore did not change the result but confirmed Nmap's assessment.

This experiment shows that OS detection can be accurate even without guess mode, but the `--osscan-guess` option is useful in situations where Nmap cannot determine the OS with confidence.

D. Experiment 4: NSE Script Scanning

Objective: The goal of this experiment is to demonstrate Nmap's ability to use the Nmap Script Engine (NSE) to gather additional information about a target, detect vulnerabilities, and perform targeted script-based attacks. NSE scripts extend Nmap's capabilities beyond standard scanning and allow enumeration, discovery, vulnerability testing, and brute-force attempts.

Setup: For this experiment, a purposely vulnerable machine was prepared on the local network (Metasploitable 2). This system exposes many open ports and insecure services, making it suitable for testing a wide range of Nmap NSE scripts. The target IP address for the experiment is 192.168.56.101.

Step 1: Service and Version Detection with `-sV` To begin the experiment, the following command was executed to identify running services and determine their versions:

```
nmap -sV 192.168.56.101
```

This scan reveals a wide range of open ports and outdated or vulnerable services running on the target. The results are shown in Figure 17.

```
C:\Users\domin>nmap -sV 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:00 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0043s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rshcd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN

Service detection performed. Please report any incorrect results at https://nmap.org
Nmap done: 1 IP address (1 host up) scanned in 12.74 seconds
```

Fig. 17. Service and version detection results for the host 192.168.56.101.

Step 2: Discovery Scripts with --script discovery

Next, general discovery scripts were run to gather additional information about the host:

```
nmap --script discovery 192.168.56.101
```

These scripts identify multicast groups, IPv6 information, and system broadcast behavior. The output is displayed in Figures 18–20.

```
C:\Users\domain>nmap --script discovery 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:14 +0100
No profinet devices in the subnet
Pre-scan script results:
targets-asn:
- targets-asn.asn is a mandatory parameter
ipv6-multicast-mld-list:
:::
device: eth1
mac: bc:dd:c2:6b:01:8b
multicast_ips:
ff02::1:ff00:0 (Solicited-Node Address)
ff02::1:ff00:0 (Solicited-Node Address)
fe80::d2ad:8ff:fe7a:a3a9:
device: eth1
mac: d9:ad:08:7a:a3:a9
multicast_ips:
ff02::1:ff7a:a3a9 (NDP Solicited-node)
ff05::c (unknown)
ff02::fb (mDNSv6)
ff02::c (SSDP)
ff05::c (unknown)
ff02::c (SSDP)
ff02::c (SSDP)
ff05::c (unknown)
ff02::1:3 (Link-local Multicast Name Resolution)
ff02::fb (mDNSv6)
fe80::3e64:cfff:feee:9c4b:
device: eth1
mac: 3c:04:cf:ee:9c:4b
multicast_ips:
ff02::1:ff00:0 (Solicited-Node Address)
ff02::2 (All Routers Address)
ff05::2 (unknown)
ff02::1:ffee:9c4b (NDP Solicited-node)
fe80::f3c7:7853:57ac:8792:
```

Fig. 18. Discovery script results (part 1) for 192.168.56.101.

```
25/tcp open smtp
_ms-sql-query: ERROR: Script execution failed (use -d to debug)
_ms-sql-info: ERROR: Script execution failed (use -d to debug)
ssl-enum-ciphers:
SSLv3:
ciphers:
TLS_DHE_RSA_EXPORT_WITH_DES40_CBC_SHA (dh 512) - F
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA (dh 1024) - F
TLS_DHE_RSA_WITH_AES_128_CBC_SHA (dh 1024) - F
TLS_DHE_RSA_WITH_AES_256_CBC_SHA (dh 1024) - F
TLS_DHE_RSA_WITH_DES_CBC_SHA (dh 1024) - F
TLS_DH_anon_EXPORT_WITH_DES40_CBC_SHA (dh 512) - F
TLS_DH_anon_EXPORT_WITH_RC4_40_MD5 (dh 512) - F
TLS_DH_anon_WITH_3DES_EDE_CBC_SHA (dh 1024) - F
TLS_DH_anon_WITH_AES_128_CBC_SHA (dh 1024) - F
TLS_DH_anon_WITH_AES_256_CBC_SHA (dh 1024) - F
TLS_DH_anon_WITH_DES_CBC_SHA (dh 1024) - F
TLS_DH_anon_WITH_RC4_128_MD5 (dh 1024) - F
TLS_RSA_EXPORT_WITH_DES40_CBC_SHA (rsa 64) - F
TLS_RSA_EXPORT_WITH_RC2_CBC_40_MD5 (rsa 64) - F
TLS_RSA_EXPORT_WITH_RC4_40_MD5 (rsa 64) - F
TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 1024) - F
TLS_RSA_WITH_AES_128_CBC_SHA (rsa 1024) - F
TLS_RSA_WITH_AES_256_CBC_SHA (rsa 1024) - F
TLS_RSA_WITH_DES_CBC_SHA (rsa 1024) - F
TLS_RSA_WITH_RC4_128_MD5 (rsa 1024) - F
TLS_RSA_WITH_RC4_128_SHA (rsa 1024) - F
compressors:
DEFLATE
NULL
cipher preference: client
warnings:
64-bit block cipher 3DES vulnerable to SWEET32 attack
64-bit block cipher DES vulnerable to SWEET32 attack
64-bit block cipher DES40 vulnerable to SWEET32 attack
64-bit block cipher RC2 vulnerable to SWEET32 attack
Anonymous key exchange, score capped at F
Broken cipher RC4 is deprecated by RFC 7465
CBC-mode cipher in SSLv3 (CVE-2014-3566)
Ciphersuite uses MD5 for message integrity
Export key exchange
```

Fig. 19. Discovery script results (part 2) for 192.168.56.101.

```
111/tcp open rpcbind
rpcinfo:
program version port/proto service
100000 2 111/tcp rpcbind
100000 2 111/udp rpcbind
100003 2,3,4 2049/tcp nfs
100003 2,3,4 2049/udp nfs
100005 1,2,3 33804/udp mountd
100005 1,2,3 35541/tcp mountd
100021 1,3,4 39219/udp nlockmgr
100021 1,3,4 42018/tcp nlockmgr
100024 1 41808/udp status
100024 1 60414/tcp status
nfs-ls: Volume /
access: Read Lookup Modify Extend Delete NoExecute
PERMISSION UID GID SIZE TIME FILENAME
drwxr-xr-x 0 0 4096 2012-05-14T03:35:33 bin
drwxr-xr-x 0 0 4096 2010-04-16T06:16:02 home
drwxr-xr-x 0 0 4096 2010-03-16T22:57:40 initrd
lrwxrwxrwx 0 0 32 2010-04-28T20:26:18 initrd.img
drwxr-xr-x 0 0 4096 2012-05-14T03:35:22 lib
drwx----- 0 0 16384 2010-03-16T22:55:15 lost+found
drwxr-xr-x 0 0 4096 2010-03-16T22:55:52 media
drwxr-xr-x 0 0 4096 2010-04-28T20:16:56 mnt
drwxr-xr-x 0 0 4096 2012-05-14T01:54:53 sbin
drwxr-xr-x 0 0 4096 2010-04-28T04:06:37 usr
nfs-statfs:
Filesystem 1K-blocks Used Available Use% Maxfilesize Maxlink
/ 7282168.0 1479184.0 5435904.0 22% 2.0T 32000
nfs-showmount:
/*
```

Fig. 20. Discovery script results (part 3).

Step 3: Vulnerability Detection with --script vuln

The next command tests the target for known vulnerabilities using the default “vuln” category of NSE scripts:

```
nmap --script vuln 192.168.56.101
```

This scan successfully identified a serious vulnerability: the vsFTPD 2.3.4 backdoor (CVE-2011-2523). Figure 21 shows the detection results, including exploit references and proof-of-concept execution.

```
C:\Users\domain>nmap --script vuln 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:01 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0068s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
ftp-vsftpd-backdoor:
VULNERABLE:
vsFTPD version 2.3.4 backdoor
State: VULNERABLE (Exploitable)
IDs: BID:48539 CVE:CVE-2011-2523
vsFTPD version 2.3.4 backdoor, this was reported on 2011-07-04.
Disclosure date: 2011-07-03
Exploit results:
Shell command: id
Results: uid=0(root) gid=0(root)
References:
http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-downloa
https://github.com/rapid7/metasploit-framework/blob/master/modules/
https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
https://www.securityfocus.com/bid/48539
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
```

Fig. 21. Vulnerability detection results: vsFTPD 2.3.4 backdoor identified.

Step 4: SSH Brute-Force Attempt with ssh-brute

Finally, a targeted brute-force script was executed against the SSH service on port 22:

```
nmap --script ssh-brute -p 22
192.168.56.101
```

The script attempted multiple default username/password pairs, demonstrating how Nmap can test login strength. The output is shown in Figure 22.

```
C:\Users\domin>nmap --script ssh-brute -p 22 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:26 +0100
NSE: [ssh-brute] Trying username/password pair: root:root
NSE: [ssh-brute] Trying username/password pair: admin:admin
NSE: [ssh-brute] Trying username/password pair: administrator:admin
NSE: [ssh-brute] Trying username/password pair: webadmin:webadmin
NSE: [ssh-brute] Trying username/password pair: sysadmin:sysadmin
NSE: [ssh-brute] Trying username/password pair: netadmin:netadmin
NSE: [ssh-brute] Trying username/password pair: guest:guest
NSE: [ssh-brute] Trying username/password pair: user:user
NSE: [ssh-brute] Trying username/password pair: web:web
NSE: [ssh-brute] Trying username/password pair: test:test
NSE: [ssh-brute] Trying username/password pair: root:
NSE: [ssh-brute] Trying username/password pair: admin:
NSE: [ssh-brute] Trying username/password pair: administrator:
NSE: [ssh-brute] Trying username/password pair: webadmin:
NSE: [ssh-brute] Trying username/password pair: sysadmin:
```

Fig. 22. SSH brute-force script attempting common credentials.

Note: Combined NSE Usage Although not demonstrated here, NSE scripts can also be combined in a single scan. For example:

```
nmap --script vuln,auth,discovery
192.168.56.101
```

This allows more complex enumeration and vulnerability assessment in one command.

Experiment Conclusion: This experiment demonstrated the power and flexibility of the Nmap Script Engine. Through service detection, discovery scripts, vulnerability scanning, and brute-force attempts, Nmap was able to gather extensive information about the target and identify serious security weaknesses. The vulnerable environment provided realistic results that clearly show how NSE can be used in real-world penetration testing and security assessments.

E. Experiment 5: Firewall/IDS Evasion Scans

Objective: This experiment demonstrates how different Nmap scan types behave when a firewall is present on the target system. By comparing results before and after firewall modifications, we can observe how firewalls filter traffic and how evasive scan types attempt to bypass these restrictions.

Setup: The target system is the intentionally vulnerable machine used in previous experiments (192.168.56.101). Two states were tested:

- Before enabling strict firewall rules (ports visible).
- After enabling firewall rules (ports filtered or blocked).

Expected Outcome: Different scan types (-sF, -sS, -sT, -sX) should return different port states depending on how the firewall handles packets.

Step 1: FIN Scan (-sF)

The FIN scan sends packets with the FIN flag. According to RFC 793, closed ports should reply with a RST, while open ports remain silent. Firewalls often treat FIN scans as suspicious and block or filter them.

```
nmap -sF -p 1-1000 192.168.56.101
```

Screenshot: FIN scan with firewall enabled All ports appear **open—filtered**, indicating that the firewall is dropping FIN packets.

```
C:\Users\domin>nmap -sF -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:51 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0059s latency).
Not shown: 988 closed tcp ports (reset)
PORT      STATE      SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
25/tcp    open|filtered smtp
53/tcp    open|filtered domain
80/tcp    open|filtered http
111/tcp   open|filtered rpcbind
139/tcp   open|filtered netbios-ssn
445/tcp   open|filtered microsoft-ds
512/tcp   open|filtered exec
513/tcp   open|filtered login
514/tcp   open|filtered shell
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.34 seconds
```

Fig. 23. FIN scan results showing all ports filtered by the firewall.

Step 2: SYN Scan (-sS)

The SYN scan (stealth scan) sends only the first step of the TCP handshake. Below are two results:

Before firewall modification: Ports visible

```
nmap -sS -p 1-1000 192.168.56.101
```

```
C:\Users\domin>nmap -sS -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:43 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0051s latency).
Not shown: 988 closed tcp ports (reset)
PORT      STATE      SERVICE
21/tcp    open      ftp
22/tcp    open      ssh
23/tcp    open      telnet
25/tcp    open      smtp
53/tcp    open      domain
80/tcp    open      http
111/tcp   open      rpcbind
139/tcp   open      netbios-ssn
445/tcp   open      microsoft-ds
512/tcp   open      exec
513/tcp   open      login
514/tcp   open      shell
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.05 seconds
```

Fig. 24. SYN scan before applying firewall rules (open ports detected).

After firewall modification: All ports filtered

```
nmap -sS -p 1-1000 192.168.56.101
```

```
C:\Users\domin>nmap -sS -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:50 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0010s latency).
All 1000 scanned ports on 192.168.56.101 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 22.52 seconds
```

Fig. 25. SYN scan after firewall rules applied (all ports filtered).

Step 3: TCP Connect Scan (-sT)

The TCP Connect scan completes a full handshake, making it easier to detect and block.

Before firewall modification:

```
nmap -sT -p 1-1000 192.168.56.101
```

```
C:\Users\domin>nmap -sT -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:42 +0100
Nmap scan report for 192.168.56.101
Host is up (0.018s latency).
Not shown: 988 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.11 seconds
```

Fig. 26. TCP Connect scan before firewall modification (open ports visible).

After firewall modification:

```
nmap -sT -p 1-1000 192.168.56.101
```

```
C:\Users\domin>nmap -sT -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:46 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0020s latency).
All 1000 scanned ports on 192.168.56.101 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 202.55 seconds
```

Fig. 27. TCP Connect scan after firewall rules applied (connections refused or filtered).

Step 4: Xmas Scan (-sX)

Xmas scans send packets with FIN, PSH, and URG flags set. Firewalls commonly block such abnormal packets, resulting in filtered output.

```
nmap -sX -p 1-1000 192.168.56.101
```

```
C:\Users\domin>nmap -sX -p 1-1000 192.168.56.101
Starting Nmap 7.98 ( https://nmap.org ) at 2025-12-03 20:52 +0100
Nmap scan report for 192.168.56.101
Host is up (0.0096s latency).
Not shown: 988 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open|filtered ftp
22/tcp    open|filtered ssh
23/tcp    open|filtered telnet
25/tcp    open|filtered smtp
53/tcp    open|filtered domain
80/tcp    open|filtered http
111/tcp   open|filtered rpcbind
139/tcp   open|filtered netbios-ssn
445/tcp   open|filtered microsoft-ds
512/tcp   open|filtered exec
513/tcp   open|filtered login
514/tcp   open|filtered shell
MAC Address: 08:00:27:19:3B:48 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.29 seconds
```

Fig. 28. Xmas scan results showing all ports filtered by the firewall.

Experiment Conclusion: This experiment clearly demonstrates how firewalls impact Nmap scan behavior:

- Without firewall rules, SYN and TCP Connect scans successfully identify many open ports.
- After enabling firewall filtering, all scan types report ports as **filtered** or **open—filtered**.
- FIN and Xmas scans are especially affected, as firewalls tend to block irregular TCP flag combinations.
- SYN scans become completely ineffective once the firewall blocks inbound SYN packets.

This experiment highlights that the effectiveness of a scan depends heavily on firewall configuration, and no single scan technique is guaranteed to bypass filtering.

VI. EVALUATION OF NMAP

Objective: This section evaluates Nmap's functionality, performance, and usability in network discovery and security auditing.

A. Functionality

Nmap excels in host discovery, port scanning, version detection, OS detection, and vulnerability scanning. The Nmap Scripting Engine (NSE) adds flexibility and automation to these tasks.

Strengths:

- Fast and efficient host discovery and port scanning.
- Accurate service detection and vulnerability assessment.
- Customizable via NSE scripts for complex tasks.

Weaknesses:

- Some scan types may be blocked or inaccurate in highly filtered environments.
- Requires raw socket access for certain features like OS detection.

B. Performance

Nmap performs well in most environments, with quick scan times for small networks. Scanning larger networks or over the internet may take longer. For example, scanning a subnet of 256 addresses took 3 seconds, while a port scan on 10 hosts with 1,000 ports took 10 minutes.

C. Accuracy

Nmap provided accurate results for host discovery and port scanning. Service version detection was reliable, though OS detection was less precise in environments with heavy filtering.

D. Usability

Nmap's command-line interface is powerful but requires some learning. NSE scripts enhance its functionality but may need basic Lua knowledge. Pre-built scripts make it easier for non-experts to use.

E. Conclusion

Nmap is a highly reliable and useful tool for network security. It is accurate, fast, and customizable, though some features may be affected by firewalls or filtering. Overall, Nmap is essential for network administrators and security professionals.

ACKNOWLEDGMENT

I would like to express my gratitude to Ing. Ján Laštinec, PhD., for creating the theme of this project. I also extend my sincere thanks to Bc. Ramon Ūrge for providing me with the opportunity to work on this project.

REFERENCES

- [1] Gordon Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*, Insecure.Com LLC, 2009.
- [2] Paulino Calderon, *Nmap Network Exploration and Security Auditing Cookbook*, Packt Publishing, 2021, ISBN: 978-1838649357.
- [3] Adnan Y. Dawod and N. Nasution, "Penetration Testing Methodology of Scanning Network using Nmap," *International Journal of Enhanced Research in Science, Technology & Engineering*, vol. 5, no. 12, 2016.
- [4] J. M. R. Redondo et al., "Towards Improving Productivity in Nmap Security Audits," *Journal of Web Engineering*, vol. 18, no. 4, pp. 353–374, 2019.
- [5] R. A. Bakar et al., "Enhancing Network Visibility and Security with Advanced Port Scanning Techniques," *Electronics*, vol. 12, no. 5, pp. 873, 2023.
- [6] Perplexity AI, *Perplexity — AI-powered answer engine for real-time web search*, <https://www.perplexity.ai/>, accessed November 29, 2025.
- [7] Liao et al., "A Comprehensive Detection Approach of Nmap: Principles, Rules and Experiments," 2020.